MEMORIA DEL PROYECTO · DESARROLLO CON IA



UN DUELO DE CARTAS FORJADO POR AGENTES DE IA



EL LICH



AXEL BLAZE

Carlos Alba Buenestado · Arturo Gómez Marín

Programación de Inteligencia Artificial (IA)



EL MAZO DE CONTENIDOS

ÍNDICE

1 Introducción y objetivos 3

2 Metodología: cascada asistida por IA 4

3 El sistema agéntico (lo importante) 5

4 Desarrollo por fases 7

4.1 · Setup y Análisis 4.2 · Diseño 4.3 · Implementación

4.4 · Pruebas (QA) 4.5 · Despliegue 4.6 · Memoria

5 Arquitectura y calidad del código 12

6 Resultado final 13

7 Conclusiones y mejoras futuras 14

8 Anexos: trazabilidad y ADRs 15

¿QUÉ ES ESTO EN UNA FRASE?

Crossover Cards es un videojuego de cartas estratégico por turnos (1 vs IA) que **no se ha programado a mano**: se ha construido dirigiendo un equipo de **19 agentes de IA** bajo supervisión humana. Esta memoria cuenta qué se hizo, cómo y por qué.

CAPÍTULO 1

1. INTRODUCCIÓN Y OBJETIVOS

Crossover Cards es un juego de cartas estratégico por turnos, 1 contra 1 (jugador frente a una IA), creado como proyecto final de la asignatura **Programación de Inteligencia Artificial**.

El juego es un homenaje **nostálgico**: personajes icónicos de las series de animación de los 2000 (Hora de Aventuras, Bob Esponja, Inazuma Eleven, Historias Corrientes...) se enfrentan en duelos temáticos. El jugador controla a un héroe singular —**El Lich** o **Axel Blaze**— y gestiona el recurso central de cada partida: la **Nostalgia**, representada por un reloj de bolsillo que se recarga turno a turno.

Pero lo importante no es el juego en sí, sino **cómo se ha hecho**. El objetivo académico real es construir software **delegando el desarrollo a un sistema de agentes de IA**, escribiendo la mínima cantidad de código manual posible, y documentando rigurosamente cada decisión, cada **prompt** y cada resultado.

OBJETIVOS ACADÉMICOS

- **Desarrollo asistido por IA:** delegar planificación, análisis, diseño, programación y pruebas a un sistema agéntico dual (nube + local), documentando todas las interacciones.
- **Arquitectura y patrones:** aplicar una arquitectura en capas limpia que aísle la lógica del juego de la presentación, con patrones de diseño bien definidos.
- **Metodología:** estructurar el avance en fases de cascada clásica, adaptadas a un plazo de 14 días.
- **Calidad (QA):** superar el 80 % de cobertura en la capa de dominio, con TypeScript **strict** sin errores y ESLint sin avisos.

OBJETIVOS DEL PRODUCTO (MVP)

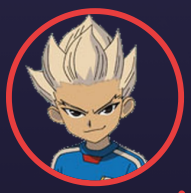
- **Partida completa** jugable de principio a fin, con victoria, derrota y empate.
- **Contenido:** 2 héroes asimétricos y un catálogo de 20 cartas (Personajes, Hechizos y Objetos).
- **Mecánicas avanzadas:** Sinergia Tribal, Momentum (remontada), Zonas dinámicas, Misiones y Aliado Invitado.
- **IA del oponente** con dos niveles de dificultad mediante el patrón Strategy.
- **Flujo de pantallas completo:** Menú → Selección de héroe → Partida → Fin.



EL LICH

Aventura Mágica · 30 PV

Hechizo Ancestral — 2 daño a cualquier objetivo.



AXEL BLAZE

Anime Combate · 30 PV

Tornado de Fuego — +2 ATQ a un aliado este turno.

CAPÍTULO 2

2. METODOLOGÍA: CASCADA ASISTIDA POR IA

Se ha optado por un ciclo de vida clásico en **cascada**, dividido en fases secuenciales y ejecutado en 14 días. Puede parecer una elección anticuada frente a las metodologías ágiles, pero **es deliberada** y responde a cómo funcionan los modelos de lenguaje.

¿POR QUÉ CASCADA Y NO ÁGIL?

Los LLM **alucinan y pierden contexto** cuando se enfrentan a requisitos ambiguos o que cambian constantemente. Al cerrar primero una documentación estable (PRD, catálogos, arquitectura), los agentes de las fases posteriores trabajan sobre un **ancla de verdad** fija. Esto reduce errores lógicos y ahorra **tokens**.

La regla de oro de todo el proyecto fue la **supervisión humana obligatoria**: ningún agente tocó el código sin presentar antes un plan que el equipo debía aprobar.

PLAN → APROBACIÓN HUMANA → EJECUCIÓN

LAS SEIS FASES DEL PROYECTO

FASE	QUÉ SE HIZO	AGENTES PRINCIPALES
1 · Setup y Análisis	Infraestructura técnica y definición del producto (PRD).	Setup-Repo, AI-Config, Project-Init, PRD, PRD-Refactor
2 · Diseño	Arquitectura, ADRs, catálogo de cartas y diseño UI/UX.	Architect, ADR-Writer, Agents-Catalog, CardCatalog, UI-Design
3 · Implementación	Traducción del diseño a código TypeScript.	Impl-Orchestrator, Domain-Dev, Phaser-Dev, AI-Opponent
4 · Pruebas (QA)	Suite de tests y auditoría de calidad del código.	Tester, QA-Reviewer
5 · Despliegue	Integración continua, README y decisión de entrega.	Deploy-Agent
6 · Memoria	Consolidación de todo el conocimiento del proyecto.	Memory-Agent

La metodología elegida condiciona directamente qué agentes existen y en qué orden actúan: cada fase de la cascada se corresponde con un conjunto de agentes especializados.

CAPÍTULO 3 · EL CORAZÓN DEL PROYECTO

3. EL SISTEMA AGÉNTICO

Esta es la parte que el enunciado valora especialmente. Todo el desarrollo se apoya en una **arquitectura dual de IA** que combina potencia en la nube con agilidad en local.

▲ NIVEL ALTO · NUBE

Google Antigravity como IDE agéntico.

Modelos potentes gratuitos (Claude Sonnet/Opus 4.6 Thinking, Gemini 3.1 Pro) para arquitectura, documentación y código complejo que requiere razonamiento profundo y contexto extenso.

▼ NIVEL BAJO · LOCAL

opencode + Ollama en la terminal.

Modelos ligeros (Qwen2.5-Coder, Qwen3) con contexto extendido a 16K para tareas mecánicas, **boilerplate** y tests repetitivos. Sin consumir cuota de la nube.

Solo se han usado modelos gratuitos, más retoques manuales puntuales.

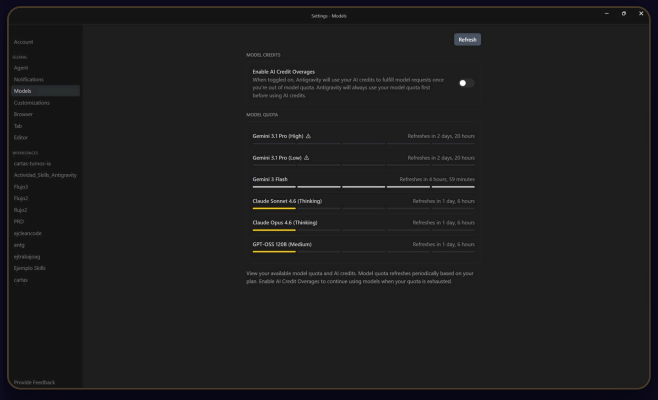


FIG. 1 *Antigravity: modelos disponibles en el IDE agéntico de nube.*

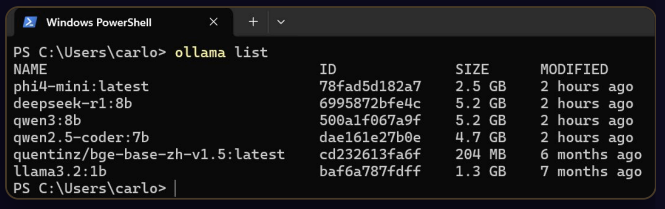


FIG. 2 *Ollama: modelos ligeros instalados para el nivel local.*

WORKFLOWS VS. CUSTOM AGENTS

Los **19 agentes** se implementan de dos formas según su necesidad de memoria:

- **Workflows (el 95 %):** archivos Markdown en `.agent/workflows/*.md` invocables con un comando (p. ej. `/prd-agent`). Sin estado persistente: cada invocación empieza limpia. Ideales para tareas acotadas.
- **Custom Agent nativo:** el **Implementation-Orchestrator**, creado desde el IDE, con **memoria persistente**. Coordinó toda la fase de implementación durante días, delegando subtareas y recordando el estado global del repositorio.

SKILLS Y RULES: EL CONTRATO DE COMPORTAMIENTO

- **Rules —** `AGENTS.md` **:** archivo raíz que todo agente lee antes de actuar. Fija el **stack**, las convenciones de nombres, la obligación de listar el directorio antes de leer archivos y de documentar cada interacción. Lo leen automáticamente tanto opencode como Antigravity.
- **Skills —** `.agent/skills/` **:** conocimiento especializado cargable bajo demanda, para no malgastar contexto. Destacan `phaser-patterns`, `solid-principles`, `clean-code` y `testing-vitest`.

GESTIÓN DE TOKENS Y ELECCIÓN DE MODELOS

Como el desarrollo dependía de modelos gratuitos con límites de cuota, se aplicó una política estricta de ahorro:

MODELO	USO RESERVADO
Claude Opus 4.6 Thinking	Arquitectura crítica y decisiones complejas (ADRs).
Claude Sonnet 4.6 / Gemini 3.1 Pro	Código sustancial, refactors profundos, documentación extensa.
Qwen2.5-Coder / Gemini 3 Flash	Tareas mecánicas, **getters/setters**, tests unitarios.
GPT-OSS 120B	Segundas opiniones puntuales sin gastar cuota primaria.

- Reparto de cuota entre los dos miembros: uno llevaba documentación, el otro código. Si un agente gastaba más del 50 % de la cuota de un usuario, la tarea pasaba al otro.
- Leer del repositorio directamente en vez de pegar bloques de código en el **prompt**.
- Modificaciones quirúrgicas (**diffs**) en lugar de regenerar archivos enteros.

TRAZABILIDAD TOTAL · DOCS/AI-LOG.MD

Cada interacción con la IA queda registrada de forma inmutable: el **prompt** emitido, la herramienta usada, el modelo exacto que lo procesó y el resultado (o archivos modificados). Es la prueba documental que exige el enunciado.

5. Política de gestión de tokens y modelos

5.1 Criterios de elección de modelo

Tipo de tarea	Modelo recomendado	Justificación
Arquitectura crítica, ADRs complejos	Claude Opus 4.6 Thinking	Mejor razonamiento profundo disponible en el free tier
Documentos largos y estructurados (PRD, arquitectura)	Claude Sonnet 4.6 Thinking	Razonamiento sólido + capacidad de estructura extensa
Refactor multi-archivo	Sonnet 4.6 / Gemini 3.1 Pro	Contexto largo y precisión en ediciones quirúrgicas
Generación de código sustancial (dominio, escenas)	Claude Sonnet 4.6 Thinking	Calidad de código TypeScript/Phaser demostrada
Documentación estructurada corta (ADRs, fichas)	Gemini 3.1 Pro	Cuota generosa, calidad suficiente para documentos acotados
Boilerplate, getters, JSDoc	Gemini 3 Flash o Qwen local	Tarea trivial, no requiere razonamiento profundo
Tests unitarios estructurados	Qwen2.5-Coder-7B (opencode)	Tarea repetitiva y estructurada; aprovecha hardware local
Segundas opiniones, alternativas creativas	GPT-OSS 120B Medium	Perspectiva alternativa sin consumir cuota de Claude/Gemini

FIG. 3 *Tabla de asignación de modelos y política de tokens del catálogo de agentes.*

CAPÍTULO 4 · LA PARTIDA, TURNO A TURNO

4. DESARROLLO POR FASES

FASE 1 — SETUP Y ANÁLISIS

CIMIENTOS TÉCNICOS Y EL CONTRATO DEL JUEGO

El proyecto arrancó montando un entorno TypeScript moderno y limpio: **Vite**, **Vitest**, **ESLint** (flat config) y **Phaser 4** como motor WebGL. Los agentes de inicialización prepararon el repositorio y configuraron los modelos locales mediante Modelfiles.

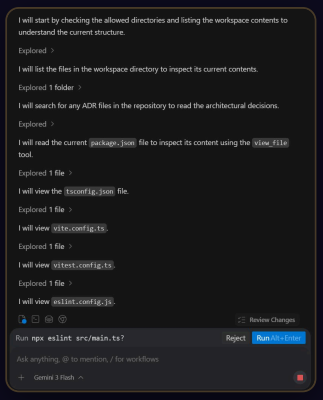


FIG. 4 *Plan de inicialización propuesto por el Project-Init-Agent.*

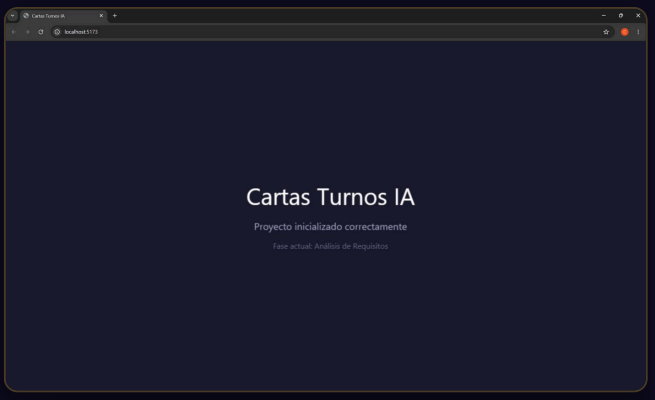


FIG. 5 *Servidor de desarrollo: "Hello World" de Phaser 4 funcionando.*

Con la base técnica lista, el **PRD-Agent** redactó un extenso Product Requirements Document: el contrato funcional del juego. Después, el **PRD-Refactor-Agent** le inyectó el ADN temático **crossover** —héroes como El Lich y Axel Blaze, y la mecánica nuclear de la **Nostalgia**—.



FIG. 6 *El PRD-Agent propone cómo estructurar los requisitos.*

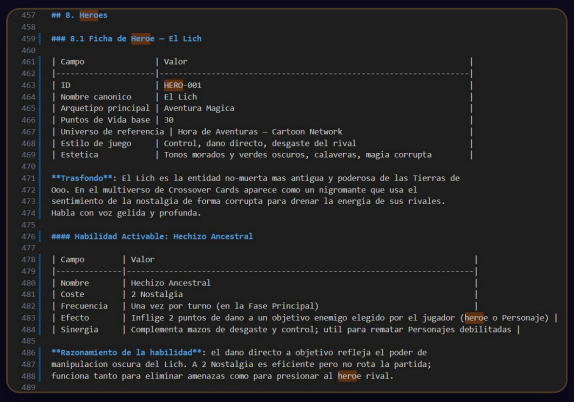


FIG. 7 *El PRD refactorizado incorpora los perfiles de los héroes.*

FASE 2 – DISEÑO

SEPARAR LA LÓGICA DE LO VISUAL

El **Architect-Agent** definió una **arquitectura en capas** que aísla por completo el **dominio** puro (las reglas del juego en TypeScript) de la **presentación** (el WebGL de Phaser). Esta separación fue clave para poder testear la lógica de forma rápida y fiable.

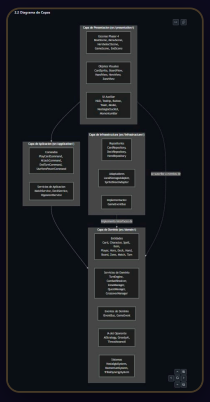


FIG. 8 *Diagrama de arquitectura en capas generado por el Architect-Agent.*

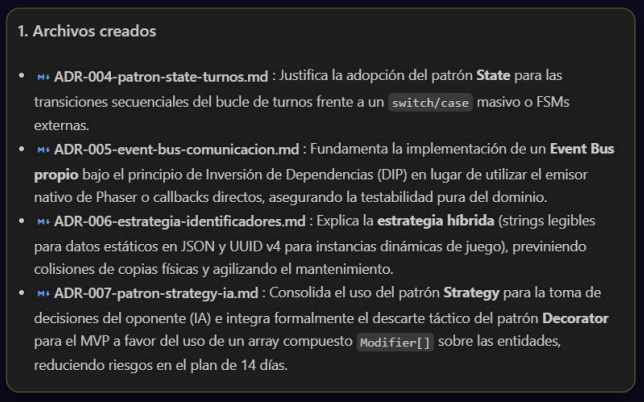


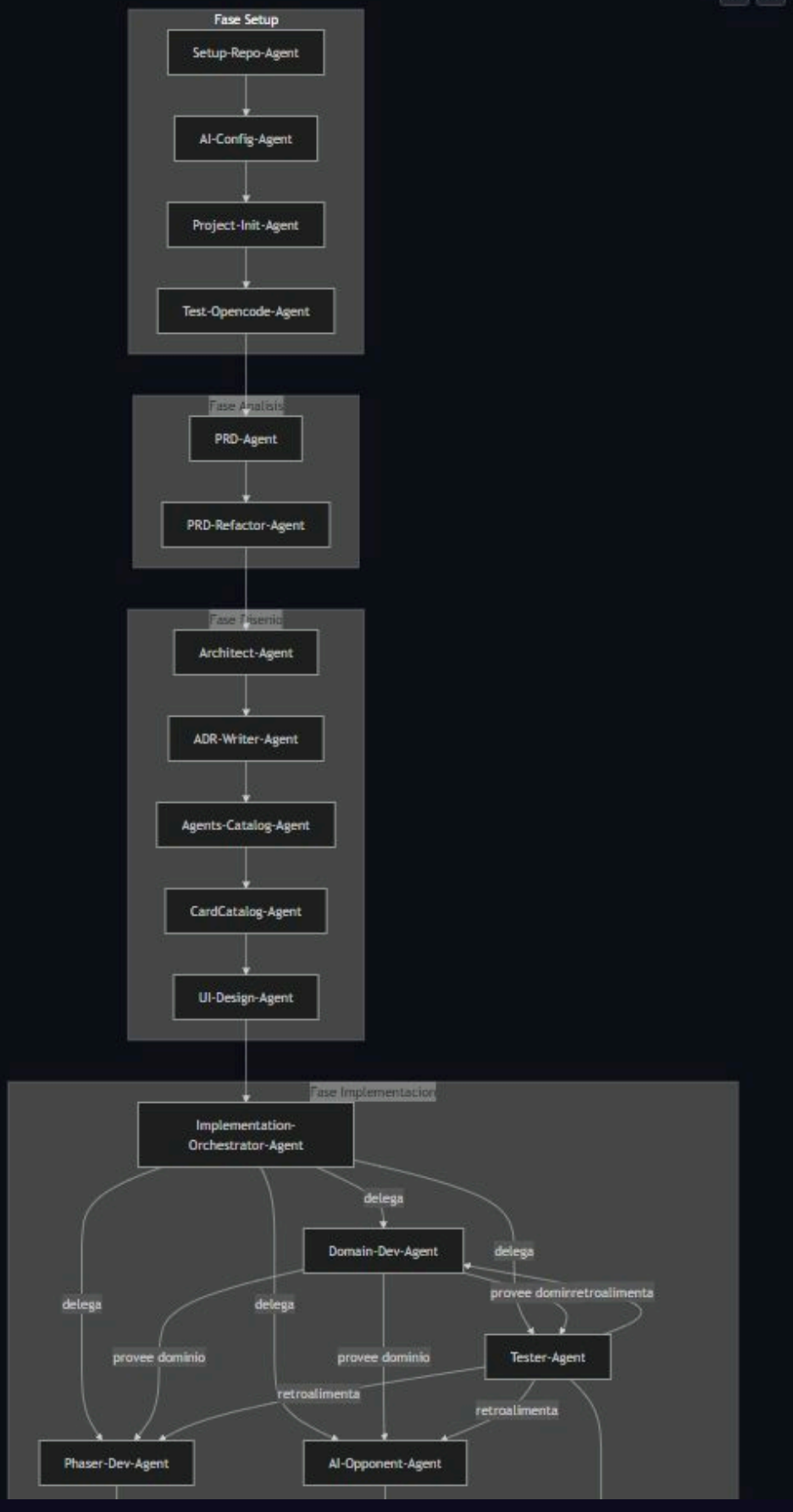
FIG. 9 *Generación automática de los registros de decisiones (ADR).*

Para que el resto de agentes (que no comparten memoria) respetaran las directrices, se adoptaron los **ADR** (Architecture Decision Records): documentos numerados que justifican cada decisión técnica (ADR-001 a ADR-008). En paralelo, el **CardCatalog-Agent** balanceó las 20 cartas en JSON y el **UI-Design-Agent** definió paletas, tipografías y **wireframes**.

4. Flujo de agentes

4.1 Diagrama Mermaid

El siguiente diagrama muestra el orden de ejecución de los agentes por fase de la cascada, incluyendo el flujo jerárquico de delegación en la fase de Implementación y el ciclo de retroalimentación del Tester-Agent.



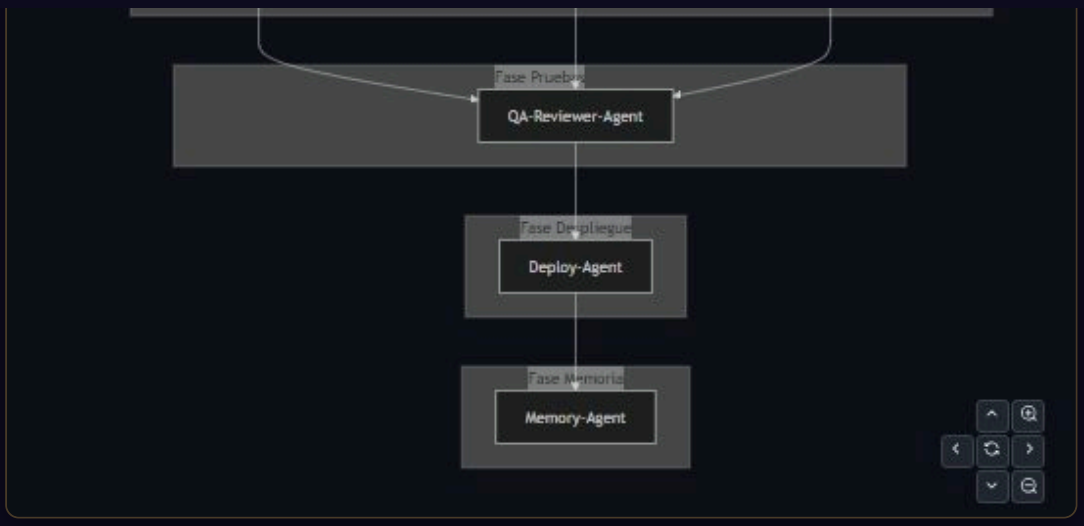


FIG. 10 *Diagrama del flujo jerárquico entre agentes durante la implementación.*

FASE 3 — IMPLEMENTACIÓN

DE LOS DOCUMENTOS AL CÓDIGO

El núcleo del proyecto. Se instanció el **Implementation-Orchestrator** (el Custom Agent con memoria) como director técnico: en vez de escribir código, dividió el trabajo en bloques (IMPL-A a IMPL-F) y delegó tareas concretas a los agentes inferiores.

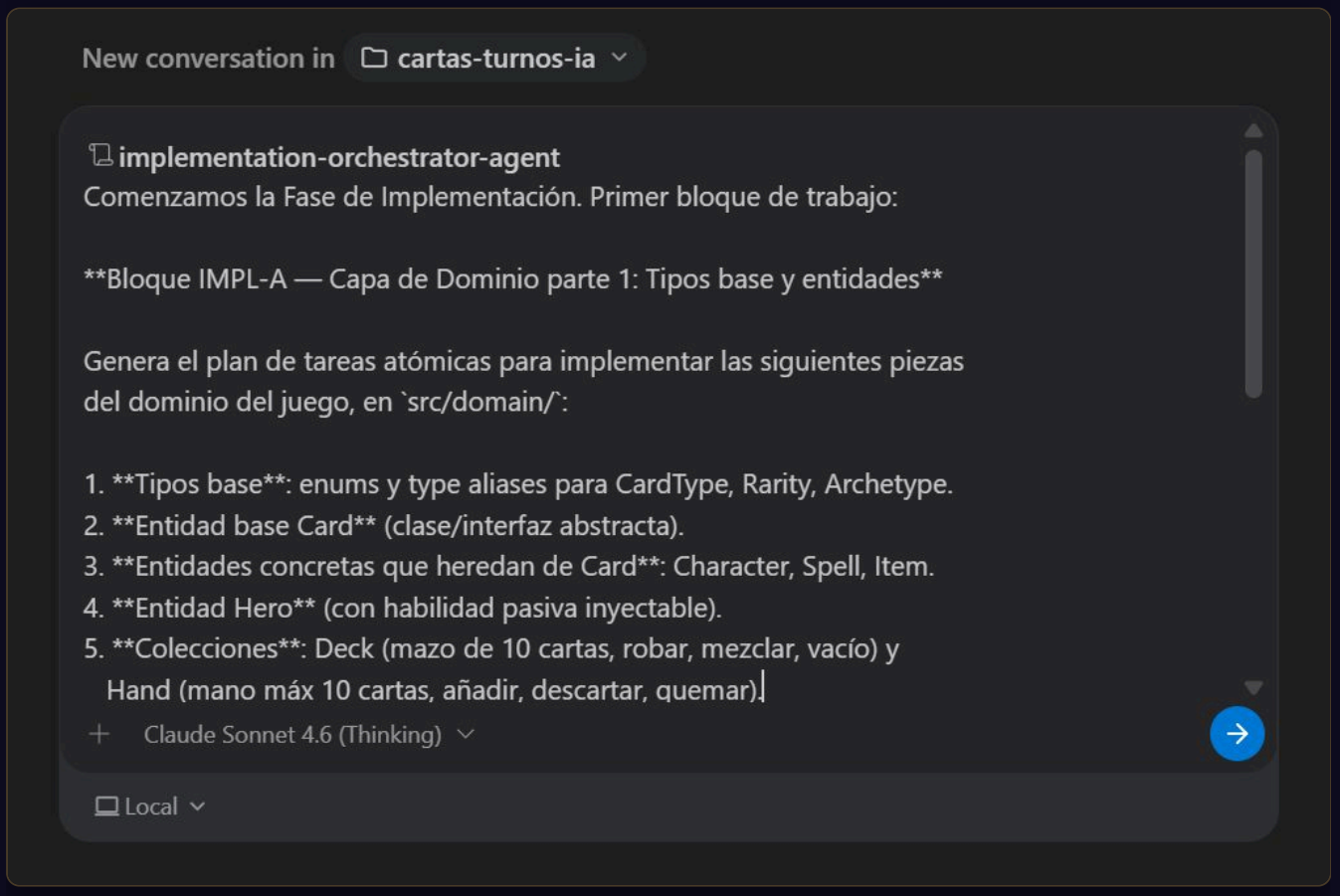


FIG. 11 *Invocación y configuración del Implementation-Orchestrator-Agent.*

Bajo esta orquestación, el desarrollo avanzó capa por capa: el **Domain-Dev-Agent** programó la lógica pura (turnos, Nostalgia, combate), el **AI-Opponent-Agent** integró el cerebro del rival, y el **Phaser-Dev-Agent** conectó esa lógica con el renderizado WebGL.



FIG. 12 *Escena de menú principal renderizada en Phaser 4.*

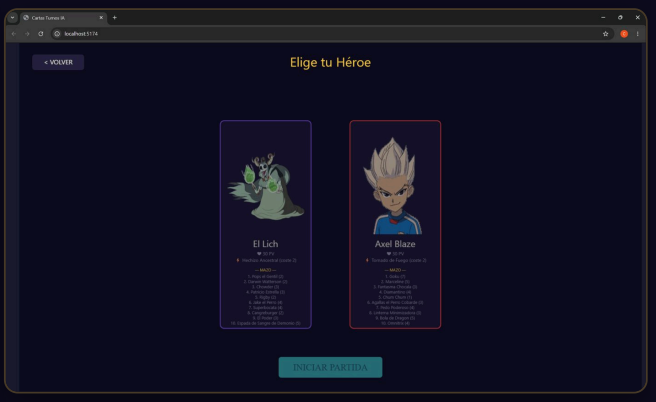


FIG. 13 *Pantalla de selección asimétrica de héroes.*

El hito central llegó con el bloque **IMPL-F**: todos los subsistemas (máquina de estados, Event Bus, UI y algoritmos de la IA) se integraron en un ciclo de partida funcional e interactivo. Después se programaron las condiciones de victoria y derrota, completando el flujo de pantallas exigido en el PRD.



FIG. 14 *El tablero de juego funcional con la interfaz completa integrada.*

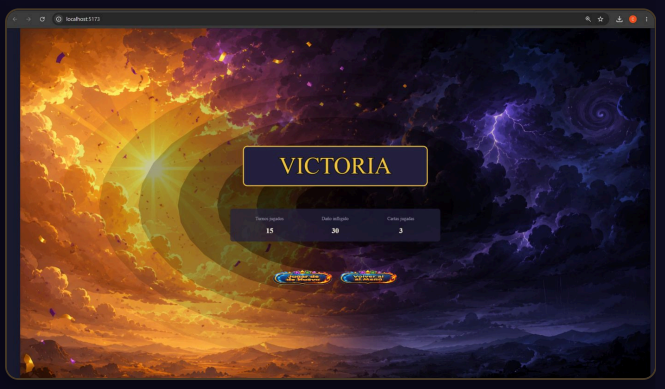
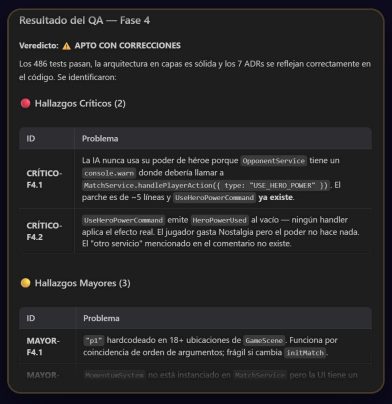


FIG. 15 *Escena de fin de partida resolviendo el resultado del duelo.*

FASE 4 — PRUEBAS (QA)

ASEGURAR LA CALIDAD

Las pruebas con **Vitest** se ejecutaron en paralelo a la codificación: el **Tester-Agent** generaba suites tras cada bloque. Para una visión global, el **QA-Reviewer-Agent** actuó como auditor senior, revisando el código contra los ADR para detectar violaciones de SOLID, **code smells** y acoplamiento.



Resultado del QA — Fase 4

Veredicto: ⚠ APTO CON CORRECCIONES

Los 486 tests pasan, la arquitectura en capas es sólida y los 7 ADRs se reflejan correctamente en el código. Se identificaron:

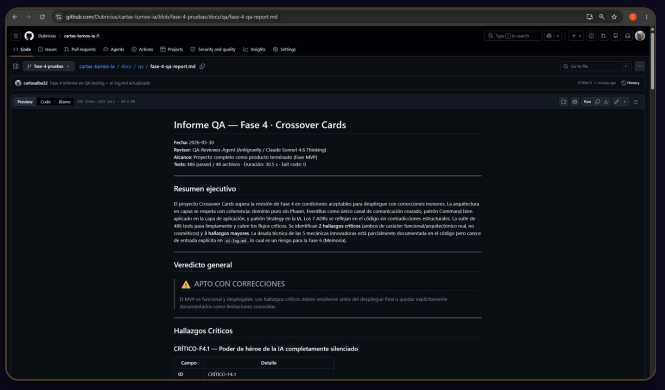
🔴 Hallazgos Críticos (2)

ID	Problema
CRÍTICO-F4.1	La IA nunca usa su poder de héroe porque `OpponentService` tiene un `console.warn` donde debería llamar a `MatchService.handlePlayerAction({ type: "USE_HERO_POWER" })`. El parche es de ~5 líneas y `UseHeroPowerCommand` ya existe.
CRÍTICO-F4.2	`UseHeroPowerCommand` emite `HeroPowerUsed` al vacío — ningún handler aplica el efecto real. El jugador gasta Nostalgia pero el poder no hace nada. El "otro servicio" mencionado en el comentario no existe.

🟡 Hallazgos Mayores (3)

ID	Problema
MAYOR-F4.1	`"p1"` hardcodeado en 18+ ubicaciones de `GameScene`. Funciona por coincidencia de orden de argumentos; frágil si cambia `initMatch`.
MAYOR-	`MomentumSystem` no está instanciado en `MatchService` pero la UI tiene un

FIG. 16 *Plan de auditoría del QA-Reviewer centrado en SOLID y acoplamiento.*



Informe QA — Fase 4 · Crossover Cards

Resumen ejecutivo

Veredicto general

APTO CON CORRECCIONES

Hallazgos Críticos

CRÍTICO-F4.1 — Poder de héroe de la IA completamente silenciado

FIG. 17 *Veredicto y reporte final de aseguramiento de calidad.*

FASE 5 — DESPLIEGUE

PREPARAR LA ENTREGA

El **Deploy-Agent** configuró un **pipeline** de **integración continua** con GitHub Actions que, en cada push, ejecuta typecheck, lint, tests y build. También redactó el **ADR-008**, que justifica entregar el proyecto como repositorio privado **sin despliegue web público**: los **assets** de los personajes están protegidos por derechos de autor.

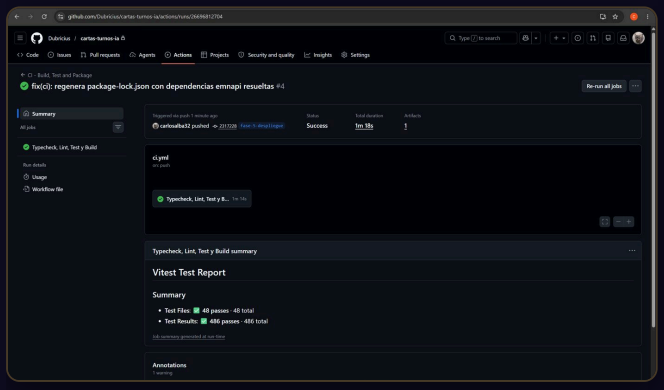


FIG. 18 *Pipeline de Integración Continua en verde en GitHub Actions.*

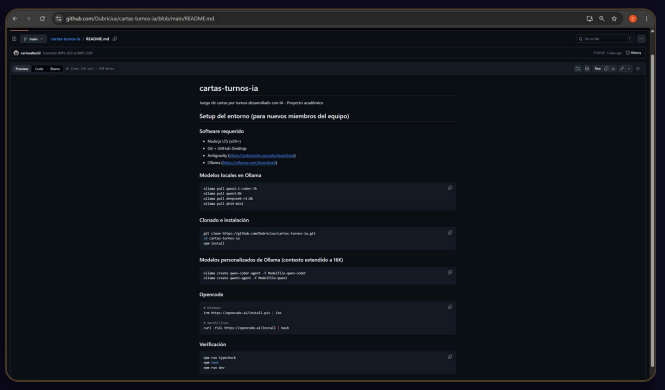


FIG. 19 *README.md renderizado, actuando como manual de instalación.*

FASE 6 — MEMORIA

CONSOLIDAR EL CONOCIMIENTO

La última fase consistió en recopilar todo el trabajo disperso. El **Memory-Agent** auditó el registro de interacciones (`ai-log.md`), reunió las justificaciones del PRD y la arquitectura, alineó las evidencias de `docs/screenshots/` y ensambló este documento. Su éxito no se mide con tests, sino por el grado de **trazabilidad** logrado entre los 19 agentes y sus resultados.

CAPÍTULO 5

5. ARQUITECTURA Y CALIDAD DEL CÓDIGO

El proyecto se sostiene sobre una **arquitectura en 4 capas** que aísla la lógica de negocio de lo gráfico. Esta decisión fue clave para el mantenimiento y para poder testear de forma robusta.

CAPA	RESPONSABILIDAD
Dominio	Lógica pura del juego (turnos, cartas, combate). Sin Phaser.
Aplicación	Comandos y servicios de orquestación.
Infraestructura	Repositorios JSON, event bus, persistencia.
Presentación	Escenas y UI de Phaser 4 (WebGL).

PATRONES DE DISEÑO APLICADOS

PATRÓN	PARA QUÉ
State	Máquina de estados de las fases del turno (inicio, robo, acción, fin).
Command	Encapsula las acciones del jugador (`PlayCardCommand` , `AttackHeroCommand`).
Strategy	Cerebro de la IA rival, con niveles intercambiables Greedy y Threat-aware.
Observer	Event Bus que desacopla el dominio de Phaser (emite eventos, la UI reacciona).
Factory / Repository	Instancia las cartas desde JSON y abstrae el acceso a datos.

MÉTRICAS REALES DE TESTING

486	48	100%	0
TESTS EN VERDE	FICHEROS DE TEST	DOMINIO (NÚCLEO)	ERRORES TS / ESLINT

CAPA	COBERTURA	OBSERVACIÓN
domain (entities, states, factories...)	**100%**	Núcleo con validación perfecta.
domain/services	94,8%	Orquestación y combate.
domain/ai	92,7%	Decisiones de la IA rival.
application/commands	92,0%	Casos de uso.
infrastructure	>97,0%	Repositorios JSON.
presentation/objects	~10,0%	Renderizado WebGL (excluido a propósito).

¿Por qué la cobertura global (74,78%) no llega al 80%? Es una decisión consciente. Testear el ciclo de repintado de **sprites** de Phaser aporta valor nulo frente a un coste altísimo. La estrategia se centró en la lógica pura, donde un fallo sería crítico. La arquitectura desacoplada garantiza que el núcleo funcione, dejando a Phaser como mero receptor visual.

CAPÍTULO 6

6. RESULTADO FINAL

Tras 14 días de desarrollo, el resultado es un **MVP funcional y jugable de principio a fin**.

PLENAMENTE JUGABLE E INTEGRADO

- **Flujo completo de escenas:** Menú → Selección de héroe → Partida → Fin (reconociendo victoria o derrota dinámicamente).
- **Núcleo de la partida:** ciclo de turnos sincronizado, recurso Nostalgia que se recarga, cartas que se arrastran al tablero pagando su coste, y ataques a esbirros o al héroe rival.
- **IA del oponente:** juega cartas y dirige ataques de forma autónoma mediante el patrón Strategy.
- **Catálogo de entidades:** los dos héroes y las cartas se cargan dinámicamente desde JSON.



FIG. 20 *Estado final del juego: partida completa funcionando en el navegador.*

TRANSPARENCIA: DESVIACIONES RESPECTO AL PRD

Las 5 mecánicas avanzadas (Sinergia Tribal, Momentum, Zonas, Misiones, Aliado Invitado) están **100% implementadas y testeadas en la capa de dominio**. Sin embargo, por la limitación temporal, su **integración visual en Phaser quedó parcial**: el motor aplica, por ejemplo, el daño del Momentum, pero la interfaz aún no muestra animaciones específicas que lo señalen al jugador.

CAPÍTULO 7

7. CONCLUSIONES Y MEJORAS FUTURAS

SOBRE EL DESARROLLO ASISTIDO POR IA

El desarrollo fue **mayoritariamente asistido por IA bajo estricta supervisión humana**. Sería falso afirmar que fue "100% automático": el valor del experimento estuvo en obligar a los agentes a operar bajo el ciclo **Planificación → Aprobación → Ejecución**, limitando la intervención manual a labores de dirección y a retoques puntuales en la integración gráfica de Phaser. Se cumplió la premisa del enunciado: **escribir a mano el mínimo código posible**.

También se demostró que la **cascada clásica es un entorno excepcionalmente favorable para los LLM**. Forzar una fase de análisis y diseño cerrada antes de programar dio a los agentes un "ancla de verdad" (los ADR y la documentación) que eliminó casi por completo las alucinaciones por pérdida de contexto.

POSIBLES MEJORAS FUTURAS

1. **Sincronización total Dominio-UI:** representar visualmente el 100% de las mecánicas ya funcionales (ranuras de Misiones, partículas de Sinergia Tribal).
2. **Expansión por JSON:** gracias al patrón Factory, añadir nuevos héroes y cartas no requiere programar TypeScript, solo alimentar archivos de datos.
3. **Multijugador (PvP):** la arquitectura de comandos y Event Bus permite sustituir la IA local por WebSockets para duelos reales por internet, sin reescribir la máquina de estados.

LA LECCIÓN EN UNA FRASE

Un equipo de agentes de IA, bien gobernado por reglas, documentación estable y supervisión humana, puede construir software real y de calidad. La clave no es la automatización ciega, sino la **orquestación disciplinada**.

ANEXOS

8. ANEXOS

ANEXO I · TRAZABILIDAD AGENTE → ARCHIVO

FASE	AGENTES	PRODUCE
1 · Setup	#001–#004	Estructura base, `package.json`, `vite.config.ts`, ADR-001/002, ai-log inicial.
1 · Análisis	#005–#006	`docs/01-prd.md`
2 · Diseño	#007–#011	Arquitectura, catálogo de agentes, catálogo de cartas, UI, ADR-004 a 007.
3 · Implementación	#012–#015	Todo el código de `src/` (dominio, aplicación, infraestructura, presentación).
4 · Pruebas	#016–#017	Suite de `tests/` y reporte de QA.
5 · Despliegue	#018	CI en `.github/workflows/ci.yml`, README, ADR-008.
6 · Memoria	#019	Este documento.

ANEXO II · ÍNDICE DE ADRS

ADR	DECISIÓN
001	Stack tecnológico: Phaser 4 + Vite + TypeScript strict.
002	Tamaño del bundle de producción.
003	Uso de AGENTS.md para guiar a los agentes.
004	Patrón State para la máquina de turnos.
005	Event Bus como comunicación entre capas.
006	Estrategia híbrida de identificadores (UUID + IDs legibles).
007	Patrón Strategy para la IA del oponente.
008	Entrega sin despliegue público (derechos de autor de los assets).

ANEXO III · REFERENCIAS

- **Repositorio:** `github.com/Dubricius/cartas-turnos-ia` (privado, acceso para `rubteacher`).

- **PRD:** `docs/01-prd.md` · **Arquitectura:** `docs/02-architecture.md`
- **Catálogo de agentes:** `docs/03-agents.md` · **Log de IA:** `docs/ai-log.md`



— FIN DE LA PARTIDA —

Carlos Alba Buenestado · Arturo Gómez Marín